

Blank Final Field :

A final field must be initialized by the developer only once.

If we declare a non static field with final modifier and if the field is not initialized at the time of declaration then it is called Blank final field.

Example :

```
class Bank
{
    final double balance; //Blank final field [Not initialized at the time of declaration]
}
```

* A blank final field cannot be initialized using non static methods.

* A Blank final field **must be initialized by the developer explicitly only once** either in non static block OR constructor body. [a) Non static Block OR Constructor body]

* A blank final field must need user-defined value till the Object creation i.e constructor body execution.

* So the final conclusion is, A Blank final field must be initialized by the developer explicitly till the object creation i.e constructor body execution in the following places

a) Non static Block
OR
b) Constructor

* A blank final field also have default value provided by new keyword.

//A blank final field cannot be initialized by default no argument constructor

```
class Bank
{
    final double balance; //Blank final field
}
```

```
public class BlankFinalField
{
    public static void main(String[] args)
    {
        Bank bank = new Bank();
        IO.println(bank.balance);
    }
}
```

//Blank final field cannot be initialized by non static methods

```
class Bank
{
    final double balance; //Blank final field

    public void setBalance(double balance)
    {
        this.balance = balance;
    }
}
```

```
public class BlankFinalField
{
    public static void main(String[] args)
    {
        Bank bank = new Bank();
        bank.setBalance(1000);
        IO.println(bank.balance);
    }
}
```

A blank final field we can initialize inside non static block

```
class Bank
{
    final double balance; //Blank final field

    {
        balance = 1000;
    }
}
```

```
public class BlankFinalField
{
    public static void main(String[] args)
    {
        Bank bank = new Bank();
        IO.println(bank.balance);
    }
}
```

Real life program on Blank final field [Engine field is blank final field]

```
package com.ravi.blank_final_field;

public class Engine
{
    private String type;
    private int horsePower;

    public Engine(String type, int horsePower)
    {
        super();
        this.type = type;
        this.horsePower = horsePower;
    }

    @Override
    public String toString()
    {
        return "Engine : [type=" + type + ", horsePower=" + horsePower + "]";
    }
}
```

```
package com.ravi.blank_final_field;

public class Car
{
    private String name;
    private int model;
    private final Engine engine; //Blank final field

    public Car(String name, int model)
    {
        this.name = name;
        this.model = model;
        this.engine = new Engine("Patrol", 1200); //Initialization is compulsory
    }

    @Override
    public String toString()
    {
        return "Car [name=" + name + ", model=" + model + ", engine=" + engine + "]";
    }
}
```

↓
Will invoke
toString() method
of Engine class

```
package com.ravi.blank_final_field;

public class BlankFinalFieldExample
{
    public static void main(String[] args)
    {
        Car car = new Car("Wagnon", 2026);
        IO.println(car);
    }
}
```

The following program explains that a blank final field also have default value

```
class Bank
{
    final double balance; //Blank final field

    {
        printDefaultValue();
        balance = 2000;
    }

    public void printDefaultValue()
    {
        IO.println("Default value is :"+balance);
    }
}
```

```
public class BlankFinalField
{
    public static void main(String[] args)
    {
        Bank bank = new Bank();
        IO.println("User Value :"+bank.balance);
    }
}
```

Relationship between the Java classes :

In java, in between the classes we have two types of relation

- 1) IS-A Relation
- 2) HAS-A Relation

1) IS-A relation we can achieve by using Inheritance concept.

Example :

```
class Vehicle
{
}
class Car extends Vehicle //IS-A relation
{
}
```

2) HAS-A Relation we can achieve by using Composition/Aggregation concept.

```
Example :
-----
class Engine
{
}

class Car
{
    private Engine engine; //[HAS-A Relation]
}
```

Inheritance [IS-A Relation]

<pre>class Developer1 { public void sum(int x, int y) { } public void sub(int x, int y) { } }</pre>	<pre>class Developer2 { public void sum(int x, int y) { } public void sub(int x, int y) { } public void mul(int x, int y) { } public void div(int x, int y) { } }</pre>
--	--

OOP says : We should always **reuse the existing code rather tahn re-writing**

For Code reusability, we should use Inheritance concept as shown in the below diagram :

```
class Developer1
{
    public void sum(int x, int y)
    {
    }

    public void sub(int x, int y)
    {
    }
}

class Developer2 extends Developer1
{
    public void mul(int x, int y)
    {
    }

    public void div(int x, int y)
    {
    }
}
```